How to Present ResilienceAI, Step by Step

BGSW AI Hackathon | Online Boutique on a local Kubernetes sandbox | about 5 minutes

The one-sentence pitch: "We built an AI helper that safely breaks parts of an online shop on purpose, watches what goes wrong, and puts everything back, so teams find weak spots before real customers do."

Step 0. Before the judges arrive (1 minute)

Check	Where	It should show
Shop opens	http://localhost:8080	Online Boutique home page with products
Dashboard opens	http://127.0.0.1:8000	Left side: green dot, "Backend connected"
Telemetry is on	Dashboard, section 1	Ingestion status: RUNNING (if not, press Start ingestion)
Nothing is broken	Dashboard, section 2	Current state: IDLE, "system is in its initial state"
Target is safe	Dashboard, section 2	Target service: cartservice (NOT frontend)
Windows stay open	Taskbar	Two PowerShell windows (shop + backend). Minimise, never close.

Put the shop and the dashboard side by side on screen, or in two tabs you can switch between quickly.

Step 1. Explain the problem (30 seconds)

Say: *"Apps look healthy until something breaks. Chaos engineering means breaking things on purpose, safely, like a fire drill. The hackathon asked for an AI that decides what to break, does it safely, and explains what it found."*

Show the shop: click a product, add it to the cart. *"This is Google's Online Boutique: 11 small services (cart, payment, shipping, product list and so on) running in our own sandbox on this laptop."*

Step 2. Show what we built (45 seconds)

Day 1 requirement (from the slides)	What we show	Proof
App selected and running end to end	The shop works: browse, cart, checkout	Shop tab
Application mapping	Map of which service calls which, built from the deployment files, with manual corrections	Dashboard section 5, tab "Service health" (Calls / Called by)
Monitoring baseline	Healthy numbers measured under steady load: 1,940 requests, 0% errors, p95 latency 26 ms	Section 4 tiles say "baseline"
First failure injection	We break a service live (Step 3)	Section 4 charts jump
Safety controls	Allowlist, validation, kill switch, rollback with verified recovery	Steps 4 and 5

Step 3. Break something live (1 minute)

#	Click / do	Say	What judges see
1	Section 2: Target **cartservice**, Fault **Service unavailability**. Tick the box. Click **Start experiment**.	*"First the safety controller checks the shop is healthy, then it takes the cart service offline."*	After ~18 s: state turns red, HOLDING (until rollback)
2	Switch to the shop tab and refresh.	*"Every page breaks, not just the cart."*	Shop error page: "Uh, oh! Something has failed"
3	Back to dashboard, section 4.	*"Live impact: 100% errors, and our monitor names the failing service from the shop's own error messages."*	Error rate 100%, Failed requests X of Y, Failing services: cartservice, red chart line
4	Point at the red box in section 2.	*"The dashboard always shows exactly which faults are in place."*	"Faults in place right now: service-down on cartservice"

Step 4. Show the safety controls (1 minute)

#	Click / do	Say	What judges see
1	Click **Kill switch (freeze current state)**.	*"The kill switch stops our agent immediately and freezes things exactly as they are."*	State: FROZEN (kill switch), press rollback
2	Try **Start experiment** again.	*"It refuses to stack a new fault on a broken system."*	Message: "faults still in place, press Rollback first"
3	Section 6: tick the box, click **ROLLBACK TO INITIAL STATE**. Wait 30-70 s.	*"Rollback restores the original state, then checks real user traffic until the shop is healthy again."*	Green: "Recovery verified N s after the restore". Refresh shop: it works.

If asked about automatic stopping: *"Every request is checked against an allowlist and limits before anything runs; we tested 8 unsafe requests and all were refused. In the agent's automatic mode, an experiment stops by itself when errors pass 10% or latency doubles. Our run records show those automatic stops (for example the product catalog outage stopped itself after about 6 seconds). For this live demo we hold the fault so you can see it."*

Step 5. Show the AI agent (30 seconds, optional)

Section 3 already shows the **latest AI plan**: a ranked table with the target, the fault, the AI's reason, its prediction, and the safety verdict. Say: *"The AI reads the service map, the healthy baseline and every past result, then ranks which experiments teach us the most. It only proposes; our safety code approves every proposal. It runs locally on the laptop's graphics card (qwen3:4b with Ollama)."*

Planning live takes about 2-3 minutes, so only click **Plan only** if you have time; its reasoning streams on screen.

Step 6. What we found so far (30 seconds)

Finding	Evidence
The cart is a single point of failure for the **whole site**: every page shows a cart counter.	Cart service down: 92% of browsing failed too
All shopping carts are **silently erased** when the cart database restarts.	Test cart went from 6 items to 0, no error shown

Finding	Evidence
Pages **hang for 5 seconds** instead of failing fast.	Redis, currency and checkout outages
A small delay is **amplified 12x**: +300 ms on the currency service made pages take 3.7 s.	Network delay experiment
A full payment outage stayed **under the 10% site-wide alarm**, so per-journey monitoring is needed.	Payment down 30 s, every order failed, site error rate only 2-8%
After a service comes back, the website takes **30-70 s** to reconnect.	Rollback recovery times

Step 7. Close (15 seconds)

Say: *"Day 1 done: app running, dependency map, baseline, live failure injection, kill switch and verified rollback. Day 2: more AI-chosen experiments, an AI-written findings report with fixes, and re-running after a fix to show the improvement."*

If something goes wrong

Problem	Do this
Dashboard says "This site can't be reached"	The backend window was closed. In a terminal: `cd C:\Users\yenis\chaos-ai; .\.venv\Scripts\python.exe -m uvicorn chaos.api:app --port 8000`
Shop won't load	In a terminal: `cd C:\Users\yenis\chaos-ai; .\open-shop.ps1`
Shop still broken after an experiment	Press Rollback in the dashboard, or run `.\chaos rollback` in the project folder
Charts are empty	Section 1: press Start ingestion and wait 5 seconds
"Port already in use"	It is already running. Just open the link in the browser.

Likely judge questions

Question	Short answer
Is this safe?	Only our own sandbox (namespace boutique) can be touched; an allowlist and limits are checked before anything runs; kill switch and verified rollback.
How does the AI decide?	It ranks experiments by user impact using the dependency map, baseline and past results, gives a reason and a prediction for each. Code validates every proposal.
Which AI model?	qwen3:4b running locally through Ollama. Claude is supported if an API key is added.
What failures can you inject?	Service outage, container crash (crash loop), and network delay.
Did you change the app?	One label fix only: the card-expiry dropdown said January twice; we fixed December.